Distributed Systems · Project 38

Parallel LLM Inference

A concept guide: every important idea in the project, what happens behind it, and why the project needs it



How to read this guide: each section explains one concept in plain language first, then shows what is working behind it, and finally explains exactly where and why it is used in Project 38. Orange boxes are everyday analogies; blue boxes are the key point to remember.

Contents

1. The big picture: what this project builds 3

2. How a language model generates text 4

3. Attention and the KV cache 5

4. Model parallelism and pipeline parallelism 7

5. Activations, RPC and serialization 8

6. Requests, step numbers and ordering 9

7. Concurrency and pipeline bubbles 10

8. Micro-batching, latency and throughput 11

9. Bounded queues and backpressure 12

10. Timeouts and failure detection 13

11. Fault recovery: restart and replay 14

12. Retries, idempotency and communication overhead 15

13. Determinism and proving correctness 16

14. What to measure and report 17

15. How everything fits together 18

16. Glossary 19

1. The big picture: what this project builds

Project 38 asks you to take a small language model (such as GPT-2) that could easily run inside a single program, and split it across **two separate worker processes** that cooperate over the network to generate text. The finished system must do four things:

- **Produce the same output** as the normal single-process model (correctness).
- **Serve several requests at once** without mixing them up (concurrency and ordering).
- **Keep working when a worker crashes**, by restarting it and rebuilding its state (fault recovery).
- **Measure the cost** of distribution: latency, throughput and bytes sent over the network.

Why do this at all? In industry, the largest models do not fit on one GPU, so companies split them across many devices. Your model is small, so the goal is not raw speed. The goal is to learn and demonstrate the distributed-systems mechanics that make split inference work reliably.

Concept area	Question it answers in this project
Transformer layers, tokens, generation	What exactly is being split and run?
KV cache	What state does each worker hold, and why does it matter?
Pipeline / model parallelism	How is the model divided between workers?
Activations, RPC, serialization	How do workers talk to each other?
Request IDs, steps, ordering	How do many requests stay separate and in order?
Concurrency, bubbles, micro-batching	How do we keep both workers busy?
Bounded queues, backpressure, timeouts	How do we avoid overload and detect trouble?
Failure detection, replay, epochs, idempotency	How do we recover from a crash safely?
Determinism, metrics, overhead	How do we prove it works and what it costs?

2. How a language model generates text

2.1 Tokens

A language model does not read letters or whole words. Text is first cut into **tokens**, small pieces such as a word or part of a word, and each token is replaced by a number (its ID). For example, “distributed systems” might become three tokens: `[distrib, uted, systems]` → `[2177, 4985, 3341]`. The model works only with these numbers.

2.2 Layers

Inside the model is a stack of identical processing blocks called **layers** (GPT-2 small has 12). First an **embedding** step turns each token ID into a long list of numbers (768 numbers for GPT-2 small). This list then passes through layer 1, layer 2, and so on. Each layer refines the representation by mixing in context from other tokens. At the top, the **LM head** turns the final representation into a score for every word in the vocabulary (about 50,000 scores), and the next token is chosen from those scores.

> **Analogy**
> A car factory with 12 workstations. The car body enters at station 1, each station adds something, and a finished car leaves station 12.

2.3 Autoregressive generation: prefill and decode

The model produces text **one token at a time**. After predicting a token, it appends that token to the text and runs again to predict the following one. Using your own output as the next input is called **autoregressive** generation. It has two phases:

- **Prefill:** the whole prompt (say 20 tokens) passes through all layers at once. This produces the first new token. Prefill handles many tokens in one go.
- **Decode:** each later step feeds in just the one newest token, passes it through all layers, and predicts the next one. This repeats until a length limit or an end-of-text token.



Figure 1. Prefill processes the whole prompt; decode adds one token per step.

The key consequence: **step n+1 cannot start until step n has finished**, because it needs the token step n produced. This dependency is the root cause of the pipeline bubble problem in Section 7.

2.4 Choosing the next token: greedy decoding

The LM head gives a score to every possible token. **Greedy decoding** always picks the highest-scoring token. Other methods (sampling) pick randomly in proportion to the

scores, which makes text more varied but different on every run. This project uses greedy decoding for testing because it is **deterministic**: the same prompt always produces the same output. That lets you compare the distributed version with the single-process version token by token (Section 13).

3. Attention and the KV cache

3.1 What attention does

Each layer contains a step called **attention**, which lets the current token look back at earlier tokens to understand context. In "The cat sat on the mat because **it** was tired", attention is how the model works out that "it" refers to "the cat".

For every token, in every layer, the model computes three vectors (lists of numbers):

Vector	Plain meaning	Computed from
Query (Q)	"What am I looking for?"	The current token
Key (K)	"What do I contain? Here is my label."	Every token, including earlier ones
Value (V)	"Here is my actual information."	Every token, including earlier ones

The current token's Query is compared with the Key of every earlier token to get a relevance score. Those scores are turned into weights, and the token takes a weighted mix of the earlier tokens' Values. In short:

```
scores  = Q_new · [K_1, K_2, ..., K_n]      # how relevant is each earlier token?
weights = softmax(scores)                    # turn scores into percentages
output  = weights · [V_1, V_2, ..., V_n]     # mix the earlier information
```

> **Key point**
> Every new token needs the Keys and Values of **all previous tokens**, in **every layer**.

3.2 The problem without a cache

Suppose the prompt has 10 tokens and you want 5 new tokens. Without a cache, every decode step recomputes K and V for the entire sequence so far, even though the K and V of old tokens never change:

Step	Tokens whose K and V are computed	Wasted recomputation
Generate token 11	10	–
Generate token 12	11	10 tokens again
Generate token 13	12	11 tokens again
Generate token 14	13	12 tokens again
Generate token 15	14	13 tokens again

The work per step grows with the length of the text, so long generations become very slow.

3.3 What the KV cache does

The **KV cache** stores the Keys and Values of every token after they are computed once. On each new step, a layer computes K and V for just the one new token, appends them to its cache, and attends over the stored ones. Work per decode step becomes roughly constant instead of growing.

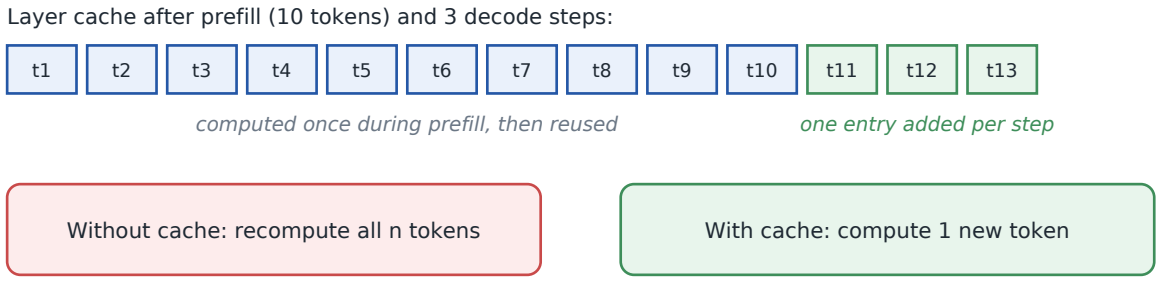


Figure 2. The KV cache keeps old Keys/Values and adds one entry per decode step.

> **Analogy**
> Adding up a long list of bills. Without a cache, every new bill makes you re-add the whole list. With a cache, you keep a running total and just add the new bill.

3.4 The cost: memory

The cache trades memory for speed. Its size is roughly:

```
cache size ≈ 2 (K and V) × layers × tokens × hidden size × bytes per number

GPT-2 small, 1 request, 512 tokens, float32:
  2 × 12 × 512 × 768 × 4 bytes ≈ 37.7 MB
```

With 8 concurrent requests that is about 300 MB, which is why caches must be deleted when a request finishes.

3.5 Why the KV cache is central to Project 38

In a single program the KV cache is just a speed trick. In this project it becomes the **main state** the whole distributed design revolves around:

- **It is split across workers.** Worker A stores the cache for layers 0–5, Worker B for layers 6–11. Neither worker has the full cache, so each must keep its part locally.
- **It keeps network traffic small.** Because each worker keeps its own cache, only the activation of the *one new token* (about 3 KB for GPT-2 small) travels between workers per decode step.
- **Every request needs its own cache.** Workers store caches in a table keyed by request ID. Mixing them up would let one user's words influence another user's output.
- **It forces ordering.** The cache grows one step at a time. Step 5 must never be applied before step 4.

- **It is lost in a crash.** The cache lives only in the worker's memory, so a crash wipes it for every active request.
- **It is rebuilt by replay.** Recovery re-runs the prompt plus generated tokens to recompute the exact same cache (Section 11).
- **It must be cleaned up.** Finished requests must free their cache or memory grows forever.

4. Model parallelism and pipeline parallelism

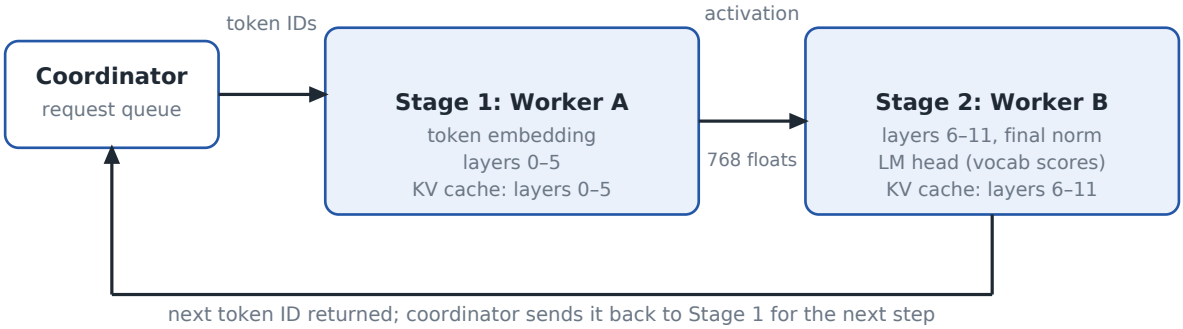
4.1 Model parallelism

Model parallelism means splitting one model across several processes or devices, so that each holds only part of it. This is different from **data parallelism**, where every device holds a full copy of the model and they handle different requests.

Strategy	How it splits work	Used when
Data parallelism	Full model copy on each device; different requests go to different copies	Model fits on one device; need more throughput
Pipeline parallelism	Each device holds a group of consecutive layers	Model too big for one device (this project)
Tensor parallelism	A single layer's matrices are split across devices	Individual layers are too big; very fast links between devices

4.2 Pipeline parallelism in this project

Project 38 uses **pipeline parallelism**: the layer stack is cut at one point and each worker becomes a **stage**. Data flows through the stages in order, like an assembly line.



Each stage runs on its own process (possibly its own machine) and talks over RPC.

Figure 3. The two-stage pipeline: what each worker holds and what flows between them.

> **Analogy**
> The car factory is split across two buildings. Building A does stations 1–6, then a truck carries the half-built car to Building B for stations 7–12.

4.3 Choosing the split point

A 6/6 split looks balanced, but Worker B also runs the final layer norm and the LM head, which multiplies by a 768 × 50,257 matrix. That is extra work, so Worker B may be slower. A slower stage limits the speed of the whole pipeline, just as the slowest station limits an assembly line. Making the split point configurable and measuring stage times lets you find a better balance (for example 7/5).

5. Activations, RPC and serialization

5.1 Activations

The output of a layer, which becomes the input of the next layer, is called an **activation** or **hidden state**. It is a tensor: a grid of numbers with shape `[batch, tokens, hidden_size]`. When Worker A finishes layer 5, the activation is what it sends to Worker B.

Phase	Shape (GPT-2 small)	Size in float32
Decode step, 1 request	[1, 1, 768]	≈ 3 KB
Decode step, batch of 8	[8, 1, 768]	≈ 24 KB
Prefill, 100-token prompt	[1, 100, 768]	≈ 300 KB
Replay after crash, 300 tokens so far	[1, 300, 768]	≈ 900 KB

These numbers drive the **transferred bytes** metric. Note how prefill and replay are much more expensive than decode. Sending float16 instead of float32 halves the bytes, at a tiny cost in precision.

5.2 RPC: Remote Procedure Call

RPC lets one process call a function that actually runs inside another process, possibly on another machine, as if it were a normal local function. Behind the scenes the RPC library packs the arguments into a message, sends it over a network connection (usually TCP), waits for the reply and unpacks it.

```
# Worker A (caller)
reply = stub.Forward(ForwardRequest(
    request_id=7, step=4, attempt=1,
    tensor=serialize(hidden)),
    timeout=2.0)
```

```
# Worker B (server)
  def Forward(self, req, ctx):
      h = deserialize(req.tensor)
      h, kv = run_layers(h, req)
      return ForwardReply(token=...)
```

Common choices: **gRPC** (typed messages, built-in deadlines), **ZeroMQ** (lightweight messaging), raw **TCP sockets** with your own message format, or **torch.distributed.rpc**. Building on gRPC or sockets shows more distributed-systems work, because you handle timeouts, retries and message IDs yourself.

> **Analogy**
> RPC is like phoning a colleague in another office and asking them to do a task. You wait on the line for the answer, but if they never pick up you need a rule for when to hang up (a timeout).

5.3 Serialization

A tensor in memory cannot be sent over a network directly. **Serialization** converts it into a flat sequence of bytes (plus its shape and data type); **deserialization** rebuilds it on the other side. Typical methods are `tensor.numpy().tobytes()` with the shape sent alongside, or `torch.save` into a byte buffer. Serialization takes CPU time on both ends, which is part of the communication overhead in Section 12.

6. Requests, step numbers and ordering

6.1 Many requests at once

A real inference server handles many users at the same time. Each user's prompt is a **request**, with its own text, its own generated tokens and its own KV cache. The system must guarantee two things:

- **Isolation:** request 7's data is never mixed with request 3's.
- **Ordering:** within one request, steps are applied in exactly the order 1, 2, 3, … with none skipped or repeated.

6.2 Message metadata

To make this possible, every message between workers carries labels, not just the tensor:

```
ForwardRequest {
  request_id : 7        # which user / prompt this belongs to
  step       : 4        # which token position in that request
  attempt    : 1        # recovery epoch; increases after each crash (Section 11)
  positions  : [23]     # token positions, needed for position embeddings
  tensor     : <bytes>  # the activation
}
```

6.3 What each worker keeps

```
kv_cache[request_id]      -> per-layer K and V tensors for this worker's layers
expected_step[request_id] -> the next step this worker will accept
attempt[request_id]       -> the current attempt number it trusts
```

When a message arrives, the worker checks it before doing any work:

Situation	Action
step == expected_step and attempt is current	Process it, append to KV cache, increase expected_step
step < expected_step (already done)	Duplicate: ignore it, or resend the stored result
step > expected_step (arrived early)	Hold it until the missing step arrives, or reject it
attempt < current attempt	Stale message from before a crash: discard

> **Analogy**
> A restaurant kitchen with order tickets. Each dish carries a table number and course number, so table 5's dessert never goes to table 7, and nobody serves dessert before the main course.

6.4 Position IDs

Transformers need to know each token's position in the sequence. When a request is at step 4 with a 20-token prompt, the new token is at position 23, not position 0. Positions must be tracked per request, and they must continue correctly during replay after a crash. Getting this wrong is a common, silent bug: the output looks plausible but no longer matches the reference.

7. Concurrency and pipeline bubbles

7.1 The bubble problem

Recall from Section 2 that each decode step depends on the previous one. With only **one request**, the two workers must take turns: while Worker B processes step n, Worker A has nothing to do, because step n+1 needs B's output first. Then A works while B waits. Each worker is idle about half the time. This idle time is called a **pipeline bubble**.

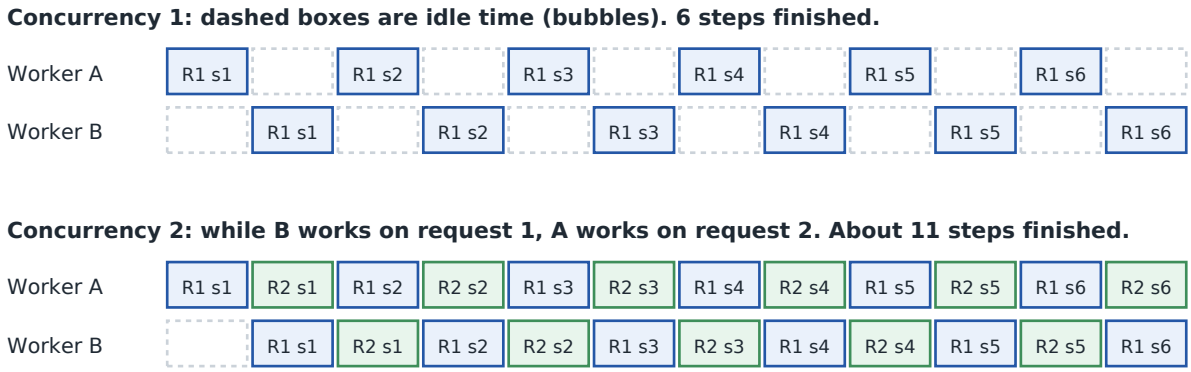


Figure 4. Same time window, one request vs. two. Each box is one stage processing one step (Rn sk = request n, step k).

7.2 How concurrency happens

Concurrency means several requests are in flight at the same time. The coordinator does not wait for one request to finish before starting another. Instead each worker runs a loop that takes the next ready message from its queue, whichever request it belongs to:

```
while True:
    msg = input_queue.get()             # next ready message, any request
    if not valid(msg): continue         # checks from Section 6
    out = run_my_layers(msg)            # uses kv_cache[msg.request_id]
    send_downstream(out)                # to Worker B, or token back to coordinator
```

Because request 2 does not depend on request 1, A can run request 2's step while B runs request 1's step. The dependency only exists *within* a request, not between requests.

More requests means more independent work available to fill the bubbles.

7.3 Concurrency levels 1, 2, 4, 8

The experiments test the pipeline with 1, 2, 4 and 8 requests running at the same time. Doubling is a convention that covers a wide range in few runs.

Concurrency	Throughput (total tokens/s)	Per-token latency for one user
1	Lowest; workers idle about half the time	Lowest; no queueing
2	Higher; bubbles mostly filled	About the same or slightly higher
4	Higher, approaching the limit	Somewhat higher; requests wait in queues
8	Levels off once both workers are always busy	Highest; more waiting per request

Throughput rises and then **levels off** (saturation). After that, extra requests only add queueing delay. Finding this point is one of the most interesting results to report.

> **Analogy**
> One car in a two-building factory leaves one building empty at all times. Several cars moving through keep both buildings busy.

8. Micro-batching, latency and throughput

8.1 Latency vs throughput

- **Latency** is how long one thing takes: time to first token (dominated by prefill) and time per output token (decode).
- **Throughput** is how much total work gets done per second across all requests, usually measured in tokens per second.

They often trade against each other. Techniques that raise throughput, such as batching and high concurrency, tend to make each individual request a little slower.

> **Analogy**
> A taxi versus a bus. A taxi gets one person there fastest (low latency). A bus is slower per person but moves far more people per hour (high throughput).

8.2 Micro-batching

Concurrency lets different requests take turns on a worker. **Micro-batching** goes further: the worker groups the pending steps of several requests into one tensor and runs its layers **once** for all of them.

```
pending steps:  R1 s5 [1,1,768]  R2 s3 [1,1,768]  R3 s9 [1,1,768]
stack       ->  batch tensor [3,1,768]
run layers once -> split result -> 3 outputs, one per request
```

Why it helps: running a layer has a fixed overhead (launching operations, reading weights from memory). For one token that overhead dominates. Processing 3 tokens in one pass costs much less than 3 separate passes, so throughput goes up.

Behind the scenes there are complications to handle:

- Each request has a different sequence length, so their KV caches have different lengths. Batched attention needs **padding** plus an **attention mask** so padded positions are ignored.
- Each request has different position IDs.
- A batching policy is needed: wait until *k* steps are ready or until a short time limit expires, whichever comes first. Waiting longer gives bigger batches but adds latency.

A more advanced version, **continuous batching**, lets new requests join a running batch as soon as others finish, instead of waiting for the whole batch to end. This is the key idea behind production systems such as vLLM, and it makes a strong extension for this project.

9. Bounded queues and backpressure

9.1 Queues

Each worker receives messages faster or slower than it can process them, so incoming messages wait in a **queue**. A queue absorbs short bursts of work and lets the sender continue without waiting for every single reply.

9.2 Why queues must be bounded

If Worker B is slower than Worker A, an unbounded queue in front of B grows forever: memory fills up, waiting time explodes, and eventually the process crashes. A **bounded queue** has a maximum size. When it is full, the producer cannot add more.

9.3 Backpressure

Backpressure is the slowdown signal that travels backwards when a downstream stage cannot keep up. In this project it flows like this:

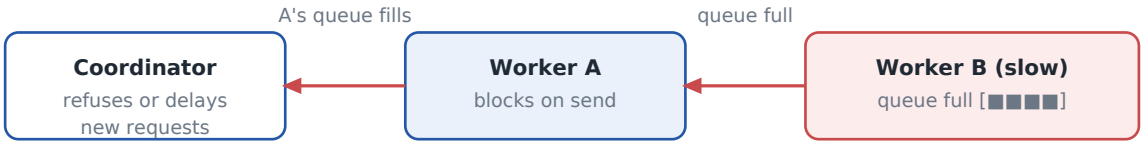


Figure 5. Backpressure: a slow stage pushes back until the system stops accepting new work.

Options when a queue is full: **block** the sender until space frees up, **reject** the new request with a “busy, try later” error, or **shed load** by dropping low-priority work. For this project, blocking between workers and rejecting at the coordinator is a clean design.

> **Analogy**
> A restaurant with 50 seats. When it is full, the host asks new guests to wait outside rather than squeezing 200 people into the room.

Key point

Backpressure turns overload into slower service instead of a crash. You can demonstrate it by adding an artificial delay to Worker B and showing that memory stays flat while new requests are delayed or refused.

10. Timeouts and failure detection

10.1 The basic difficulty

In a distributed system, one process cannot see inside another. If Worker B stops replying, Worker A cannot tell whether B has **crashed**, is just **very slow**, or the **network** between them is broken. From outside, all three look exactly the same: silence. This is one of the central facts of distributed computing.

10.2 Timeouts

A **timeout** is a maximum time the caller will wait for a reply. Every RPC call and every request has a deadline. If no reply arrives in time, the caller assumes something failed and starts recovery.

Timeout choice	Effect
Too short	False alarms: a slow but healthy worker is treated as dead, causing needless restarts and replays
Too long	Slow detection: real crashes leave users waiting a long time before recovery begins
Sensible rule	A few times the normal step time, e.g. normal decode ≈ 50 ms, timeout ≈ 1–2 s; prefill needs a longer limit

10.3 Heartbeats

A **heartbeat** is a small “I am alive” message a worker sends at regular intervals, for example every 500 ms, to a supervisor. If several heartbeats in a row are missed, the worker is declared failed. Heartbeats detect a crash even when no request is currently in flight, while RPC timeouts detect problems during active work. Using both gives faster and more reliable detection.

Analogy

A friend who always replies within minutes goes silent for an hour, so you assume something is wrong and check on them. You cannot be sure they are in trouble, but waiting forever is worse.

10.4 Failure model

A good report states its **failure assumptions** explicitly. This project assumes **crash-stop failures**: a worker either works correctly or stops completely. It does not assume Byzantine failures, where a worker sends wrong results on purpose. It does need to handle **delayed messages**, because a message from before a crash may still arrive afterwards

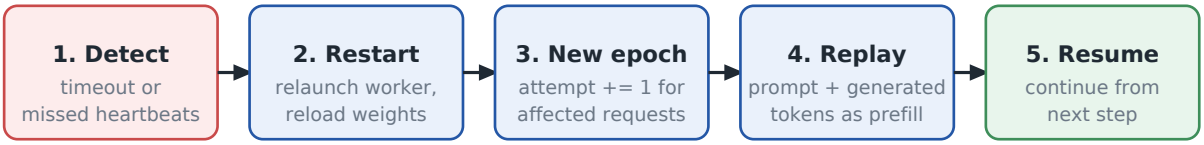
(Section 11.3).

11. Fault recovery: restart and replay

11.1 What is lost in a crash

Weights are not lost: they are on disk and can simply be reloaded. What is lost is the **KV cache** held in the crashed worker's memory, for every active request. Without it, the worker cannot continue a request from the middle. The coordinator, however, still knows each request's prompt and every token generated so far, because it received each token as it was produced. That record is enough to rebuild everything.

11.2 The recovery procedure



Replay recomputes the exact same Keys and Values, so the rebuilt cache is identical to the lost one.

Figure 6. The five steps of crash recovery.

Replay works because the model is **deterministic**: the same tokens at the same positions always produce the same Keys and Values. Example:

```
Request 3: prompt = 20 tokens, generated so far = 15 tokens, then Worker B crashes.

Replay input  = 20 prompt tokens + 15 generated tokens = 35 tokens, positions 0..34
Worker A      : clears its old cache for request 3, recomputes layers 0-5 for 35 tokens
Worker B      : (fresh) computes layers 6-11 for 35 tokens -> cache rebuilt
Output        : token 36, exactly what it would have been without the crash
```

Worker A also redoes its part even though it did not crash. This keeps both stages' caches consistent with the same attempt. An optimization is for A to keep its cache and resend only the activations, which is worth comparing in the report.

11.3 Stale outputs and epochs

After a crash, old messages may still be travelling: a result the old Worker B sent just before dying, or a request that was sent, timed out, and later arrives anyway. These are **stale outputs**. If accepted, they could add a token twice or corrupt a cache.

The fix is an **epoch** (also called an attempt or generation number). Every recovery increases the request's attempt number, and every message carries the attempt it belongs to. Receivers discard any message with an older attempt, or with a step that is already complete.

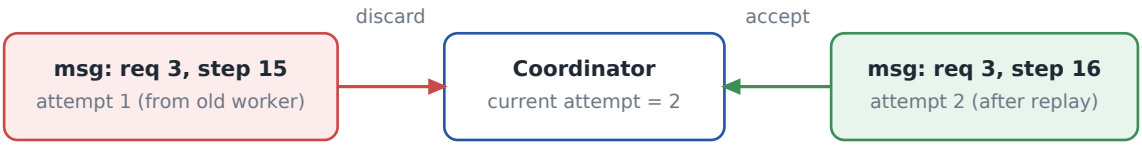


Figure 7. Epoch numbers let the system reject leftovers from before the crash.

Analogy

After a security incident, a company reissues ID cards with a new version number. Old-version cards are refused even if they look genuine.

12. Retries, idempotency and communication overhead

12.1 Retries

When a call times out, the natural response is to **retry** it. But a timeout does not mean the work was not done. Maybe Worker B did process step 5 and only the reply was lost. A blind retry would make B process step 5 a second time, appending a duplicate entry to its KV cache.

12.2 Idempotency

An operation is **idempotent** if doing it twice has the same effect as doing it once. Steps become idempotent through the labels from Section 6: when a worker receives step 5 again for the same request and attempt, it sees that step 5 is already done and either ignores the message or returns the stored result. Retries then become safe.

Analogy

Pressing a lift button five times does not call five lifts. Extra presses change nothing.

Common pitfall

Some RPC libraries retry automatically. If those retries do not carry your step and attempt IDs, they can create duplicate steps. Turn off automatic retries, or make sure every retry is detectable.

12.3 Communication overhead

Every time an activation moves between workers, the system pays costs that a single process never pays:

- **Serialization**: turning the tensor into bytes, and back on the other side.
- **Transmission**: network latency (even on one machine, a local socket adds delay) plus bandwidth time for large messages.
- **Waiting**: the RPC round trip, during which a worker may sit idle.

This is why the pipeline is usually **slower than one process for a single request**. That is not a failure. It is the expected cost of distribution, and the report should explain it.

Ways to reduce it include sending float16 activations, micro-batching (one message carries several requests), and overlapping communication with computation.

> **Analogy**
> Splitting a factory across two buildings means trucking cars between them. The driving time is pure overhead that a single building would not have.

Where time goes in one decode step	What to measure
Worker A compute (layers 0–5)	Timestamp before/after the layer loop
Serialize + send + deserialize	Timestamps on both sides of the RPC; bytes per message
Worker B compute (layers 6–11 + LM head)	Timestamp before/after
Return token + coordinator bookkeeping	Round-trip time minus the parts above

13. Determinism and proving correctness

The strongest claim this project can make is: **the distributed system produces exactly the same text as the single-process model, even after a crash**. To check this you need a reference and a deterministic setup.

13.1 The reference implementation

First build a single-process version with your own manual layer loop and KV cache, and confirm it matches the library's built-in `generate()` with greedy decoding. Every later version is compared against it.

13.2 What to compare

- **Token IDs**: should match exactly with greedy decoding. This is the main test.
- **Logits**: may differ by tiny floating-point amounts, because splitting and batching can change the order of additions. Compare within a tolerance such as 1e-4, not for exact equality.
- **After a crash**: the final token sequence of a run with a forced crash must equal a crash-free run.

13.3 Why floating-point results can differ slightly

Floating-point addition is not perfectly associative: (a + b) + c can differ from a + (b + c) in the last digits. Batching and padding can change the order of operations, producing tiny differences. Very rarely, two tokens have almost equal scores and the tiny difference changes which one greedy decoding picks. If that happens, it is worth documenting rather than hiding.

13.4 Correctness test matrix

Test	Passes when
Manual loop vs. generate()	Identical token IDs
In-process split (Stage 1 then Stage 2)	Identical to reference
Two processes over RPC, 1 request	Identical to reference
Many concurrent requests, shuffled/delayed messages	Each request identical to its own reference
Micro-batching on	Identical tokens; logits within tolerance
Crash Worker B at a chosen step	Final text identical to crash-free run
Duplicate and stale messages injected	No duplicated or missing tokens

14. What to measure and report

Metric	Definition	How to collect it
Time to first token (TTFT)	Time from submitting a prompt to receiving the first generated token	Timestamp at submit and at first token
Per-token latency	Average time between consecutive generated tokens	Timestamps per token; report mean and p95/p99
Throughput	Total tokens generated per second across all requests	Tokens / wall-clock time at concurrency 1, 2, 4, 8
Transferred bytes	Bytes sent between workers, per token and per request	Count message sizes; split prefill vs. decode
Recovery time	Time from crash to generation resuming	Crash timestamp to next accepted token
Replay cost	Extra tokens recomputed because of recovery	Replay length; plot vs. crash position
Stage utilization	Fraction of time each worker is busy	Busy time / total time per worker

Experiments worth running

- Single process vs. pipeline with one request: shows communication overhead.
- Pipeline at concurrency 1, 2, 4, 8: shows bubbles being filled and the saturation point.
- Micro-batching on vs. off: shows the throughput gain and the latency cost.
- Different split points: shows load balance between stages.
- Crashes at early vs. late steps: shows recovery cost growing with sequence length.
- Artificially slow Worker B: shows backpressure keeping memory bounded.

> **Tip**
> Log raw timestamps, byte counts and token IDs to files, and generate every graph from those files with a script. Then every number in the report can be traced back to its source.

15. How everything fits together

Here is the life of one request from start to finish, with the concept at work in each step:

#	What happens	Concept
1	User submits a prompt; coordinator assigns request_id 7 and checks capacity	Bounded queue, backpressure
2	Prompt is tokenized and sent to Worker A with step 0, attempt 1	Tokens, message metadata
3	Worker A embeds tokens, runs layers 0–5, stores K/V in kv_cache[7]	Pipeline stage, KV cache
4	Activation is serialized and sent to Worker B by RPC with a deadline	Activations, RPC, serialization, timeout
5	Worker B checks step/attempt, runs layers 6–11 and the LM head, stores K/V	Ordering, idempotency, KV cache
6	Greedy decoding picks the next token; it is returned to the coordinator	Determinism
7	Coordinator records the token and sends it to Worker A as step 1	Autoregressive decode
8	Meanwhile, other requests use the idle stage; ready steps may be batched	Concurrency, bubbles, micro-batching
9	Worker B crashes at step 15; the call times out and heartbeats stop	Failure detection
10	B is restarted, attempt becomes 2, prompt + 15 tokens are replayed	Recovery by replay, epochs
11	A late attempt-1 message from old B arrives and is discarded	Stale outputs
12	Generation continues; final text matches the crash-free reference	Correctness
13	Request finishes; both workers delete kv_cache[7]	Memory management

The whole project in one paragraph

The model is a stack of **layers** split across two workers (**pipeline parallelism**). Workers pass **activations** over **RPC**, labelled with **request IDs, step numbers and attempt numbers**. Each worker keeps its own **KV cache** per request as its state. **Concurrency** and **micro-batching** fill **pipeline bubbles**, trading **latency** for **throughput**, while **bounded queues** provide **backpressure**. When a worker fails, **timeouts and heartbeats** detect it, **replay** rebuilds the cache, **epochs** discard **stale outputs**, and **idempotency** makes retries safe. **Greedy decoding** makes the output deterministic so correctness can be proven, and measurements reveal the **communication overhead** that distribution costs.

16. Glossary

Term	Meaning
Activation / hidden state	The tensor output of a layer, passed to the next layer; sent between workers.
Attempt / epoch	A number that increases after each recovery; used to reject messages from before a crash.
Attention	The step in each layer where a token gathers information from earlier tokens using Q, K and V.
Autoregressive	Generating one token at a time, feeding each output back in as input.
Backpressure	A slow downstream stage forcing upstream stages to slow down, preventing overload.
Bounded queue	A waiting line with a maximum size.
Concurrency	Several requests in progress at the same time.
Continuous batching	Letting new requests join a running batch as soon as others finish.
Crash-stop failure	A failure where a process simply stops; it never sends wrong results.
Decode	Each generation step after prefill, processing one new token.
Determinism	Same input always gives the same output; achieved here with greedy decoding.
Greedy decoding	Always choosing the highest-scoring next token.
Heartbeat	A periodic “I am alive” message used to detect failures.
Idempotency	Doing an operation twice has the same effect as doing it once.
KV cache	Stored Keys and Values of earlier tokens, so they are not recomputed every step.
Latency	Time taken by one request or one token.
LM head	Final layer that turns the hidden state into a score for every vocabulary token.
Micro-batching	Combining several requests’ pending steps into one tensor processed in one pass.
Model parallelism	Splitting one model across several processes or devices.
Pipeline bubble	Idle time in a pipeline stage while it waits for another stage.
Pipeline parallelism	Model parallelism where each worker holds a group of consecutive layers.
Prefill	The first pass over the whole prompt, producing the first new token.
Replay	Re-running the prompt plus generated tokens to rebuild a lost KV cache.

Term	Meaning
RPC	Remote Procedure Call: calling a function that runs in another process.
Serialization	Converting a tensor into bytes for sending, and back again.
Stale output	A leftover message from before a recovery that must be discarded.
Throughput	Total work done per second, e.g. tokens per second across all requests.
Timeout	The maximum time to wait for a reply before assuming failure.
Token	A small piece of text represented by a number.